CHAPTER 24

Language Reference: Basic Types and Expressions

So far, we've given a narrative look at how Ruby works. In this chapter, we're going to use more of a reference structure to discuss Ruby's syntax as it concerns basic types and basic expressions. In the next chapter, we'll cover syntax relating to objects and classes.

Source Layout

Ruby is a line-oriented language. Ruby expressions and statements are terminated at the end of a line unless the parser can determine that the statement is incomplete, such as if the last token on a line is an operator or comma, or if there is still an open delimiter such as a parenthesis, square bracket, or curly brace. A backslash at the end of a line also tells Ruby to continue the expression onto the next line.

```
# no backslash '\' needed -- ends with an operator
d = 4 + 5 +
    6 + 7

# no backslash '\' needed -- has an unclosed parenthesis
3 = (4 + 5 +
     6 + 7)

# backslash '\' needed -- ends with a number
f = 8 + 9   \
    + 10
```

A semicolon can be used to separate multiple expressions on a line.

```
a = 1
b = 2; c = 3
```

Comments start with # and run to the end of the physical line. Comments are ignored during syntax analysis.

A line starting with `=begin` starts a multi-line comment, which ends with a line starting with `=end`. Lines in-between are ignored by Ruby and may be used to comment out sections of code or to embed documentation. The `=begin` and `=end` markers can not be indented, they must start in the first column of a line, like this: